Portfolio

UX/UI designer

Zhangyong

2026个人项目

Hi，我是张雍。
一个正在把研究、AI和产品设计连接起来的UX设计师。

我喜欢研究那些“看起来只是小问题,但实际会影响体验节奏”的地方：用户为什么会反复点错、为什么会在页面里来回跳、为什么一个AI产品生成完之后很难继续用下去。它们看起来方向不同，但都在解决同一件事：让复杂的体验，变得更清楚、更轻、更容易继续。

项目经历

美团校企合作 | 移动界面动线冗余度研究

2025.11 – 2026.04

- 参与典型业务任务的用户测试与事后回顾，通过行为观察与半结构式访谈，分析用户在任务中的停顿、返回、误操作与路径偏移等体验问题。
- 将用户主观感知转化为可识别的交互行为模式，结合操作日志与专家判断完成行为抽象，并通过脚本处理 Session 数据、重建真实操作路径。将异常结果进一步映射至信息组织、交互反馈与路径引导等界面问题，支持后续设计诊断。

南京照明集团 | 智慧照明B端平台功能设计

2025.04 – 2025.06

- 负责南京照明集团 B 端平台的前期业务调研与新增功能流程框架梳理，完成部分关键流程原型及配套视觉资产交付。
- 完成基础风格 LoRA 模型训练，并参与搭建生成式视觉工作流，用于概念视觉生成、风格探索与方案快速迭代。



个人网站：yongzhang.online

邮箱：zhangyong@njfu.edu.cn

手机/微信：130-7543-4960

南京，中国

南京林业大学

工业设计工程 · 硕士　　2024 – 至今

重庆文理学院

机械工程 · 本科　　2020 – 2024

作品分区

我挑出4个作品案例，这些案例是我多样化设计能力的代表。

---

01 Mirror | AI换装与个人风探索

用户洞察 产品策略 交互设计 AI 体验

02 UI Trace | 逆向设计系统插件

交互设计 需求拆解 设计系统

03 ChatGPT再设计 | 聊天管理

交互设计 多端设计

04 美团 | 移动界面动线冗余度研究

用户研究 数据分析 体验评估

把一次生成，
变成可以继续修改的穿搭方案。

用户可以从一件已有单品、一套喜欢的穿搭或过去保存的方案开始，只修改想改的部分，不用每次重新生成。



项目背景

用户在为旅行、演出或重要活动准备穿搭时，通常要在几个App之间来回找参考，再回想自己有哪些衣服，试着把这些信息组合在一起。

这个过程需要花很多时间。过去搭配成功的衣服也很难继续使用， 下一次遇到相似的场合，用户往往还要重新找灵感、重新搭配，

解决方案

Mirror是一款桌面端AI穿搭概念应用。 它把灵感库、电子衣柜、A生成和穿搭方案管理放在同一个产品中。

用户可以从灵感或已有单品开始搭配，查着生成效果，修改其中的单品或局部细节，再把结果保存下来，留到以后继续使用。

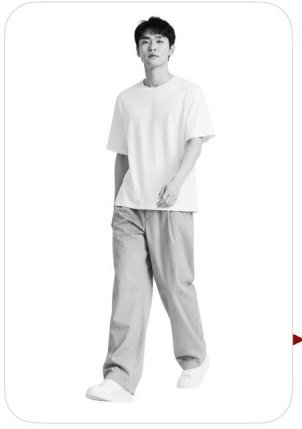
我的角色与产出

作为独立设计师，我完成了产品定义、4位用户访谈、信息架构、交互设计和视觉系统，并交付了一套包含25张核心界面的高保真原型与可交互前端demo。

我保存的不是一张图片，而是一套可以继续修改的穿搭。

Mirror 将所有可复用的生成结果一起保存，让用户之后仍然能够替换单品、修正细节，并从原方案继续。

只保留一张图片，会丢失哪些信息？

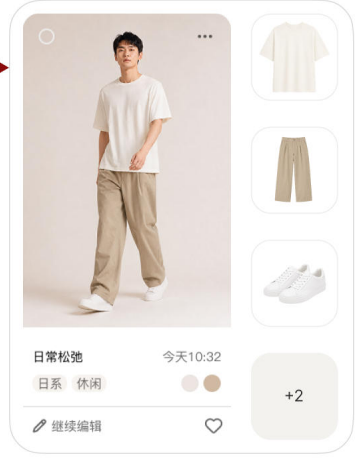


只保留结果，无法继续编辑

图片只留下最终画面，想换掉其中一件衣服时，用户往往只能重新输入并再次生成。

保留生成关系，可以继续修改

穿搭方案会保留生成过程中的模特、单品、灵感和结果。用户可以从原方案继续，只修改发生变化的部分。



一套穿搭方案包含什么？



对应哪些功能？

我的穿搭

管理我的所有穿搭方案，可以继续编辑于复用。

我的衣柜

沉淀我的单品库，一键穿搭更高效。

计划与分析

按场景/时间规划穿搭，并与他人分享灵感。

用户更需要修改、找回与复用。

我们邀请了 4 位目标用户进行了用户访谈。

1. 了解用户平时怎么找搭配、记录穿搭，以及过程中最麻烦的地方。
2. 了解用户怎么看待 AI 试穿结果，哪些问题会直接影响他们的信任。
3. 了解用户会怎么保存、找回、修改和分享满意的穿搭方案。

“我只想把上次那套的裤子换一下。”

记忆线索

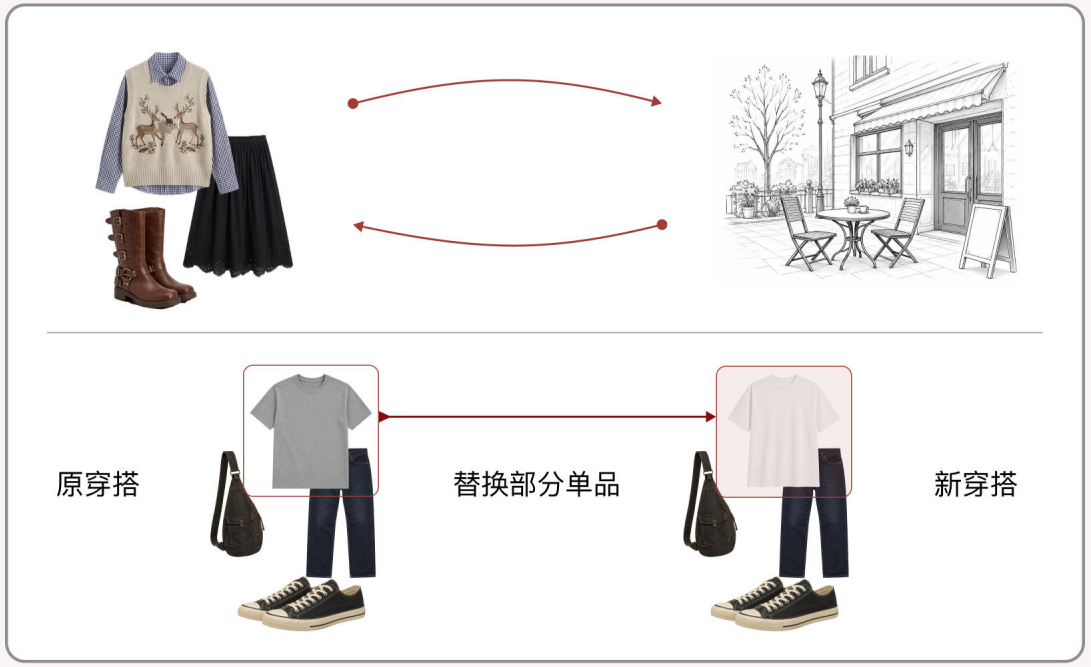
用户很少按照日期回忆一套穿搭，更多时候会想到“黑色外套”，或者“上次去音乐节穿的”。相比时间，单品和具体场景更接近用户真实的记忆方式。

方案复用与局部调整

用户找到一套曾经满意的穿搭后，不是重新搭，只想改一点，只换掉裤子、外套或鞋子。相比继续生成新方案，能够在旧方案上调整更符合她们的实际习惯。

断点与焦虑感

用户愿意让AI进行生成，但不能失控，不希望原本的图案、颜色和版型被随意改变。当结果出现偏差时，她们更希望保留当前方案，只修正有问题的部分。



自由不一定易上手，
所以我放弃了无限画布。

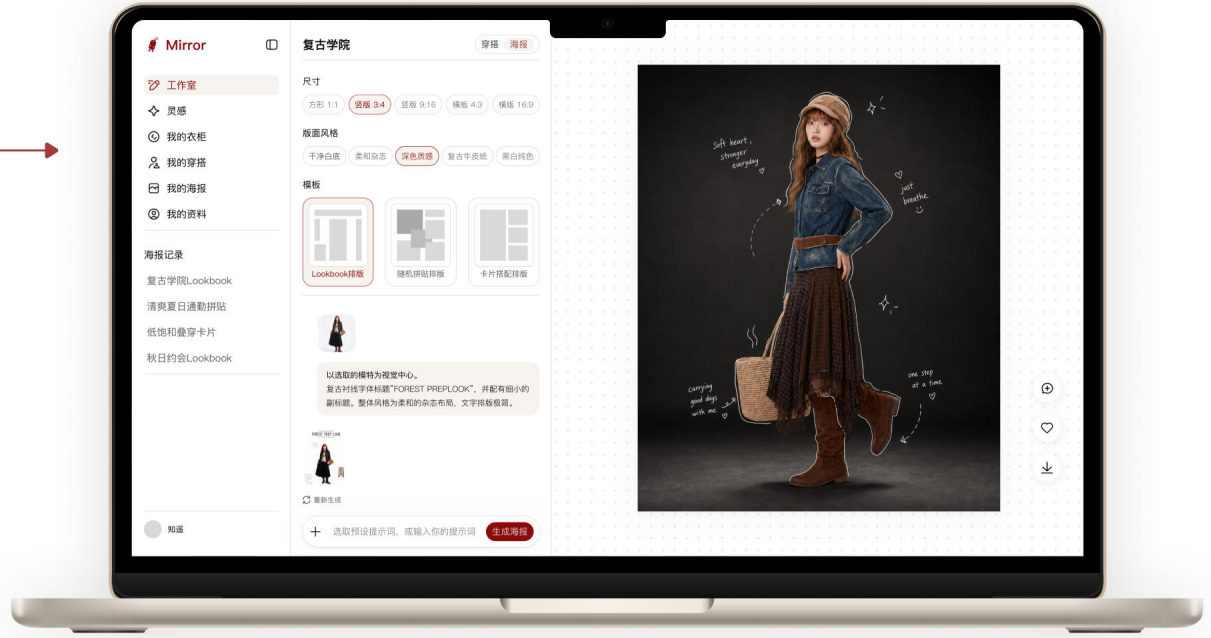
早期设想：用无限画布自由组合素材

最初的工作室接近一块数字情绪板。

用户可以把模特、单品和参考图片放在画布上，自由调整位置和组合关系，再根据这些内容生成穿搭。



无限画布虽然提供了更高的自由度，却增加了额外的操作成本。用户需要先完成画布整理操作，这些行动与穿搭判断无关，却分散了注意力。



这次调整保留了什么？

用户仍然可以组合素材，比较结果和替换单品。被取消的是需要主动整理和管理的画布。进入工作室后，用户可以直接把注意力放在衣服和生成结果上。

从不同起点开始，都会汇入工作室。

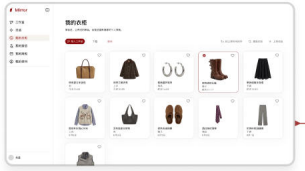
有些用户先找到一张喜欢的参考图，有些先决定要穿哪件衣服，也有只想继续修改过去的方案。
Mirror 保留三种起点带入的不同内容,再将它们汇入统一的生成与修改流程。

左栏：导航区
快速浏览历史生成记录。

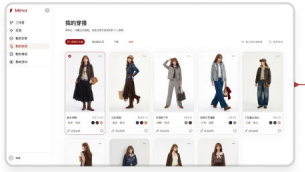
中栏：输入区
放入单品、模特与提示词。

右栏：结果区
展示生成结果，支持局部修改。

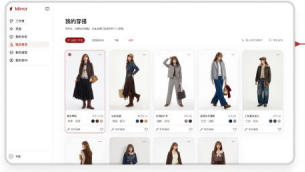
从灵感开始
带入完整的风格、配色与提示词。



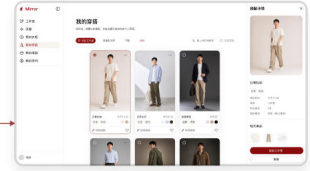
从单品开始
先锁定单品，再补充其他内容。



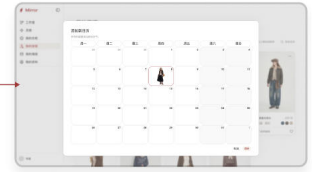
从旧方案开始
保留原模特、单品组合与生成结果。



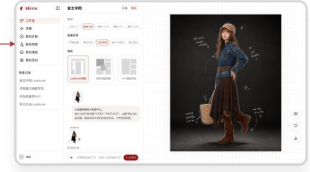




保存方案



加入计划



分享输出

生成 ⟶ 比较 ⟶ 修改 ⟶ 保存

把“换一件衣服”和“修正生成图片”分成两种操作。

用户说“我想改一下”时，可能在说两件完全不同的事。

一种是觉得搭配里的某件衣服不合适，另一种是搭配已经满意，但 AI 把细节生成错了。

为什么需要分成两种操作?

替换单品代表用户改变了搭配选择，框选修改代表用户保留当前搭配，只处理 AI 没有完成好的部分。

将两种操作分开以后，用户能够预先知道结果会发生多大变化，也能避免只想修改一个细节，却让整张图片重新生成。

替换单品：改变穿搭方案里的衣服

当用户觉得某件单品不合适时，可以从衣柜中选择另一件进行替换。

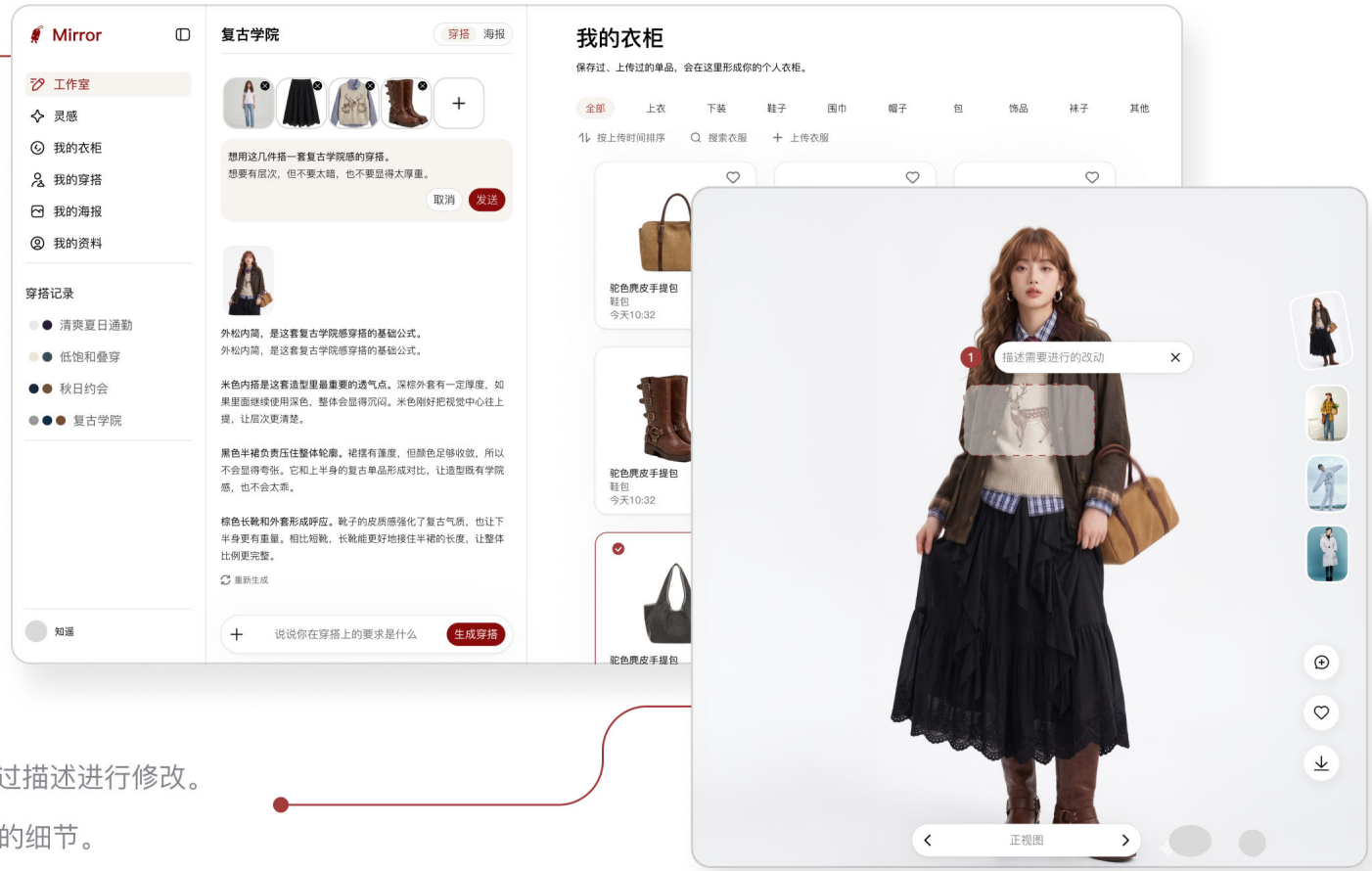
其他单品会继续保留，系统会根据新的组合重新生成结果。

这次修改改变的是穿搭方案本身。

框选修改：处理当前图片里的局部问题

当用户只想处理AI生成中的局部错误时，可以选中问题区域，再通过描述进行修改。

例如，去掉多余的配饰，恢复衣服原来的图案，或者修正某个错误的细节。



从一件单品，
找回它出现过的所有历史穿搭。

用户很难通过日期想起过去的穿搭，却会清楚记得其中的一件衣服。
因此，我把衣柜中的每件单品和它出现过的历史方案关联起来。用户打开一件熟悉的外套，就能看到过去用它完成的所有搭配。

用一件衣服查找过去的方案

每件单品的详情页都会显示包含它的历史穿搭。用户不用回忆具体日期，也不用逐张查看保存的图片，可以从一件记得的衣服开始查找。

方案详情会显示整套穿搭的组成

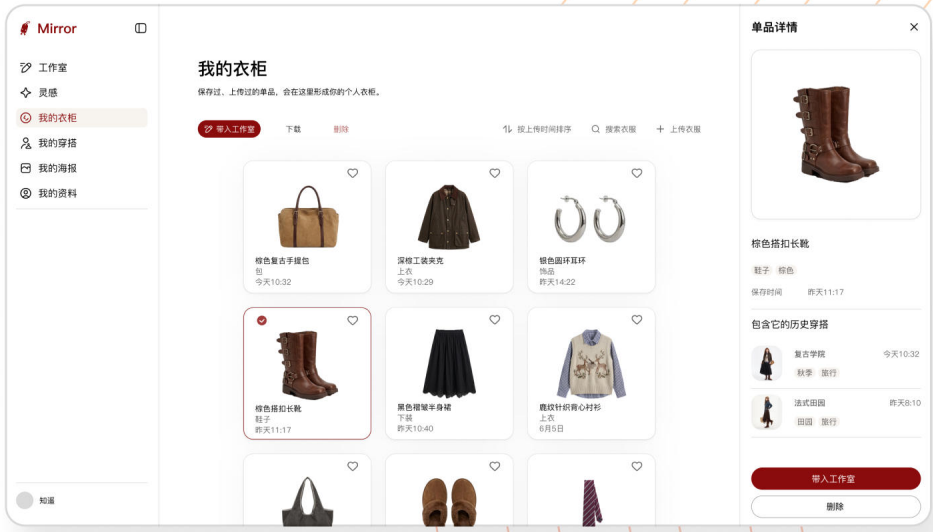
一套历史方案中，用户可以同时看到最终效果图和其中使用的全部单品。

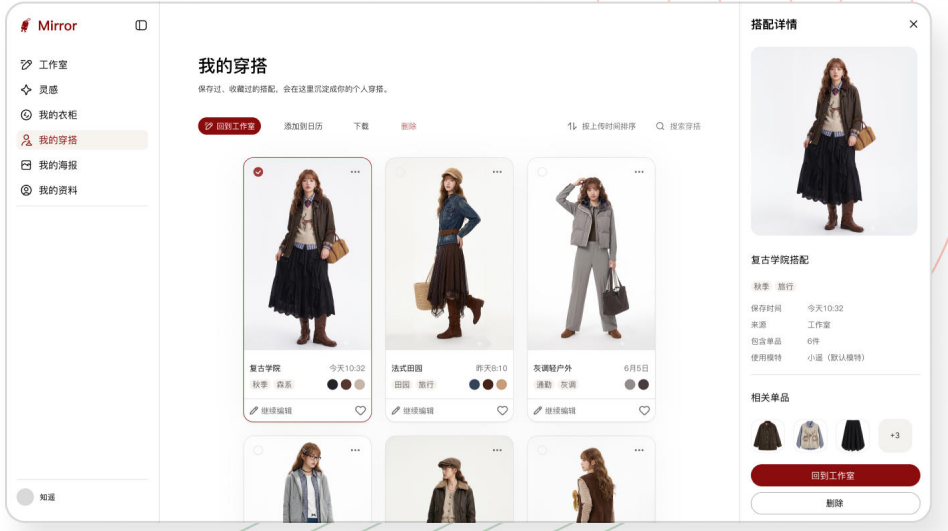
通过这些信息，用户能快速确认这是不是自己想找的那套搭配。

复制旧方案，再完成新的版本

用户可以复制任意一套历史方案，并把副本重新放入工作室。

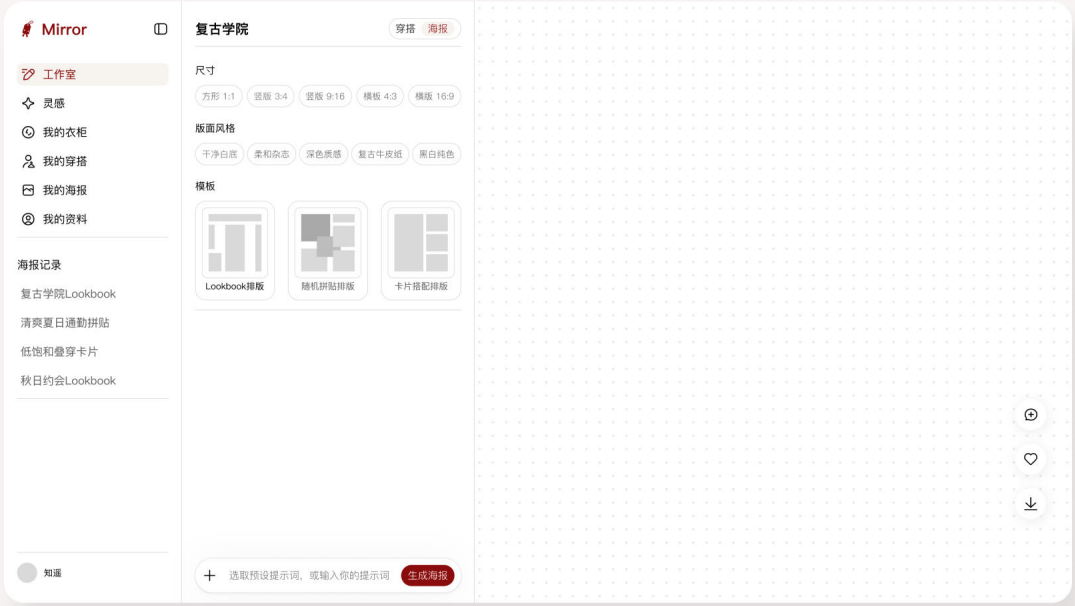
原来的模特、单品和搭配关系都会保留。用户可以在副本中更换部分内容，让这套穿搭适应新的活动或新的衣服。

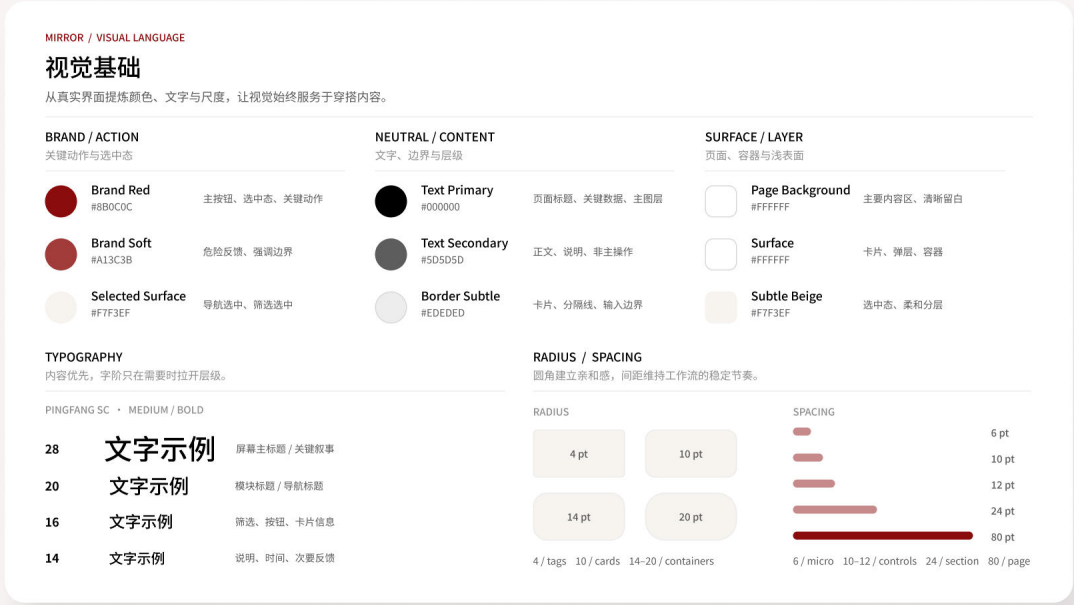


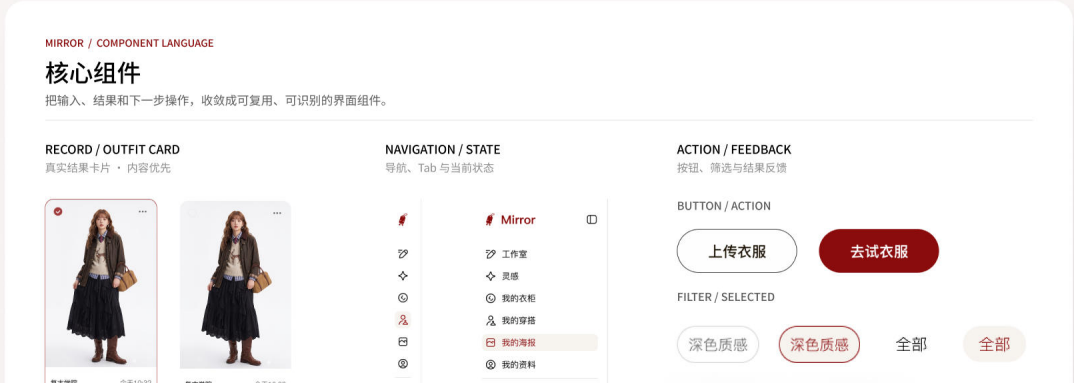


让界面保持安静，
把注意力留给穿搭本身。

通过克制的配色、清晰的层级和稳定的结构，
让用户在输入、结果与调整之间自然流动，专注于穿搭的表达与探索。







中性视觉与内容聚焦

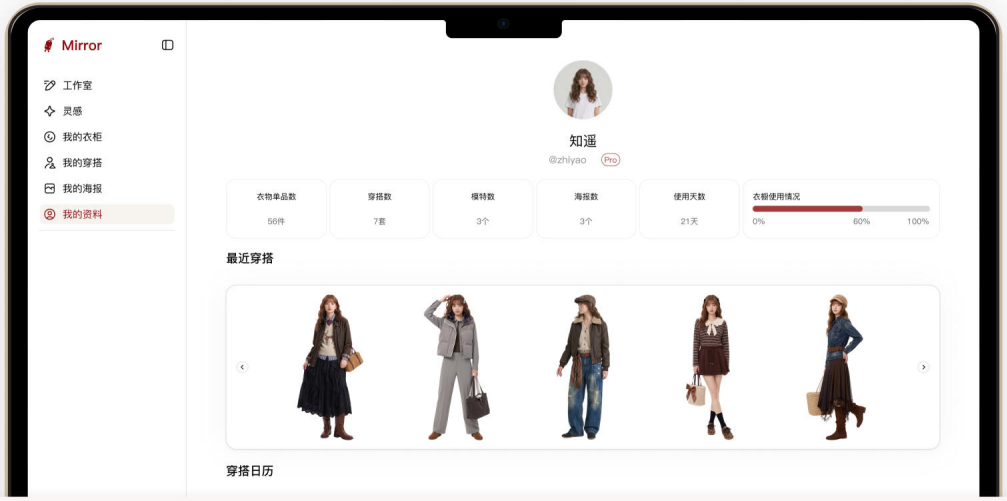
以中性色为主，弱化界面存在感，
让穿搭结果成为视觉焦点，提升沉浸感。

功能色与状态识别

深红用于关键行动与选中状态，
建立清晰的视觉反馈，快速理解可操作点。

做完Mirror后，
我留下了四个关于AI创意工具的判断

完成这个项目后，我将关键设计选择收束为四条判断，分别对应方案保存、错误修正、输入展示与最终决策。



生成结果的可持续使用

AI 生成不应停留在一次性输出。保留模特、单品与结果之间的关系，才能支持后续修改与场景复用。

修改方式的分级反馈

整体搭配调整与局部生成错误需要不同的操作方式，让用户在修改前明确结果会发生多大变化。

输入信息的持续可见

原始输入需要与生成结果保持并置，帮助用户判断问题来自输入选择还是生成过程。

用户决策权的最终保留

AI 可以提供方案和执行修改，但适配场景、审美判断与最终结果仍由用户确认。

在不确定中建立可控的创作流程，让AI成为可靠的创意伙伴。

提取网页真实状态，生成设计系统草稿。

我做了一个浏览器和Figma的组合工具,能把别人网页里的颜色、字体和组件状态直接抓下来，自动排版成能修改的设计草稿。

这是什么项目

这是一套逆向提取设计规范的辅助工具。用户在浏览器里采集成熟网页，系统会自动导出数据，并在Figma中排版成可以直接修改的规范草稿。

给谁解决什么问题

为需要参考成熟视觉规则的设计师，以及需要建立视觉基准的独立开发者而做。手工截图吸色太慢了，他们需要把网页里的散乱特征，直接变成能复用的规则。

我的角色与产出

我完成了从概念到最终上线的全部工作。包括梳理双端交互流程、设计界面，并借助AI跑通了代码开发与双端原型的走查验证。



Web端：采集真实网页状态

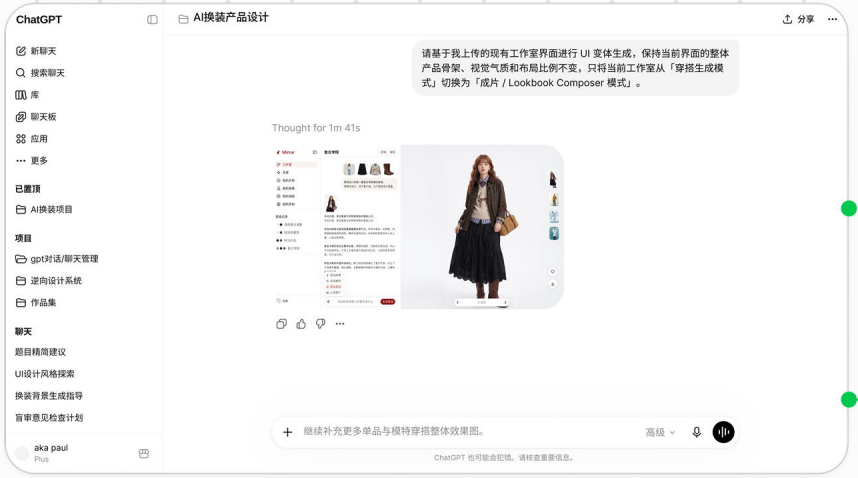
Figma端：生成设计系统草稿



从成熟产品找规律，代替手工重复操作。

这个工具的起点是我自己的工作痛点。

每次做新项目找参考需手动整理过于繁琐，因此我想要把重复的特征自动理出来。



过去：手工重复，效率低且不稳定



目标：自动提取，沉淀课复用的设计系统规律



寻找稳定的系统规律

以ChatGPT 为例，它功能完善但界面非常干净。用来测试这套工具，就能清楚看出插件工具是只能吸几个散碎色值，还是真能理出整个产品的设计规律。

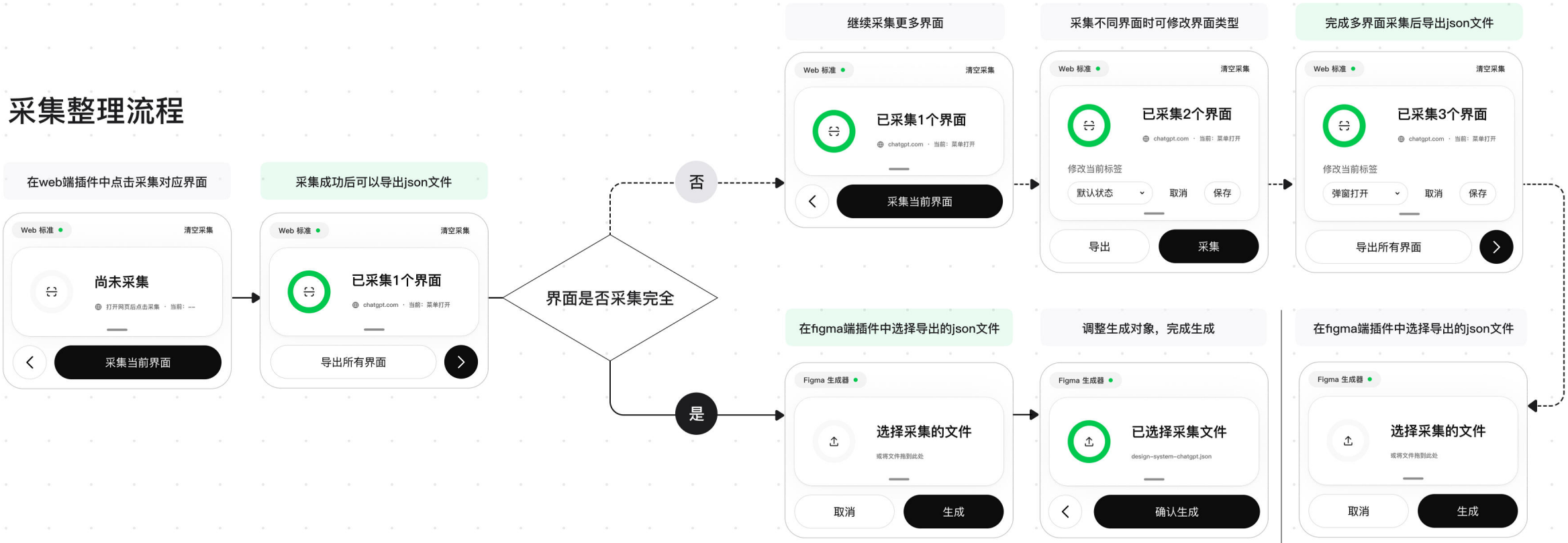
告别手工整理

在没有系统学习设计系统整理前，想要整理一个成熟产品的视觉风格，更多是截图整理素材、颜色再重新画icon。页面一多，根本分不清哪些是规范。**所以我想把这个过程自动化。**

点几下，把网页设计整理进Figma。

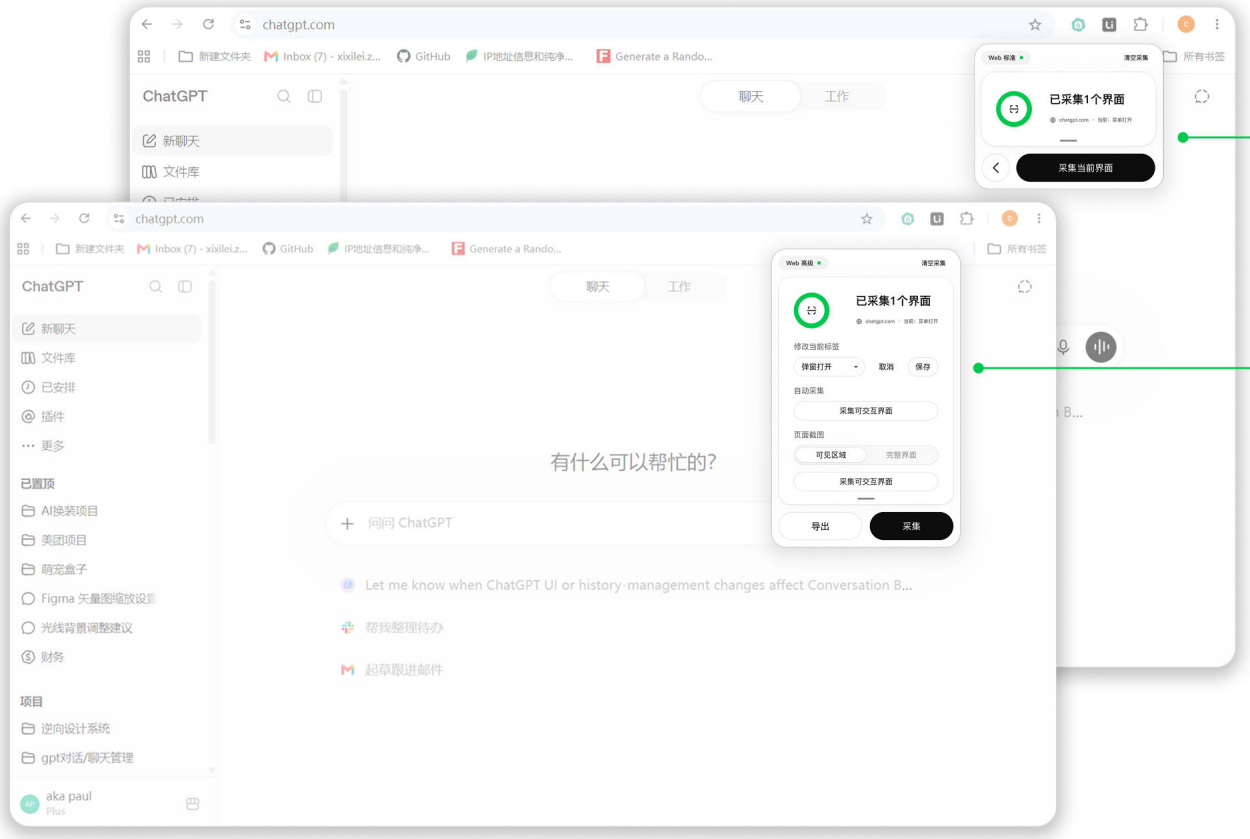
Web端负责采集与整理，Figma 端负责生成与编辑；两端通过 JSON 串联，把零散的网页参考转化为可继续使用的设计内容。

采集整理流程



不挡住网页，只在需要时展开。

在网页里抓取状态时，视线焦点都在网页上。
我没有做复杂的全屏插件，而是只保留了一个很小的悬浮操作卡片。



按需展开的选项

面板默认只展示当前域名和状态，只有用户点击下拉热区时，才会展开更多的可操作信息。

连续采集不被打断

将原本繁琐的表单填写变为自动识别，用户可以持续抓取，也可以再碰到关键状态时再顺手检查标签是否正确。

避免面板抢戏

当用户点开一个复杂的菜单准备抓取时，最怕面板挡住视线。我设计了一个合理尺寸的ui尺寸，默认状态下只保留最核心的操作按钮。

清晰的状态确认

抓取完成后，圆环和数字会直接给出明确反馈。**不用转移视线也能知道操作已经成功。**

根据抓取进度，让下一步操作成为视觉重心。

我没有把所有功能都堆在首页,而是根据操作到了哪一步来显示对应的按钮。不用专门看说明书,最显眼的按钮就是下一步该干的事。

聚焦的操作行为

抓取时采集按钮成为操作重点，抓取完成后恢复平级的导出与采集。你可以选择继续抓，也可以随时离开。

顺利成章的离开

当用户准备离开网页端时，操作重点换为导出所有界面按钮。贴合了用户从单点聚焦到批量打包的真实心理。



尚未采集任何界面

在尚未采集界面时，展开热区也无法交互。此时只提供唯一的下一步操作：采集当前界面。

已采集1个界面

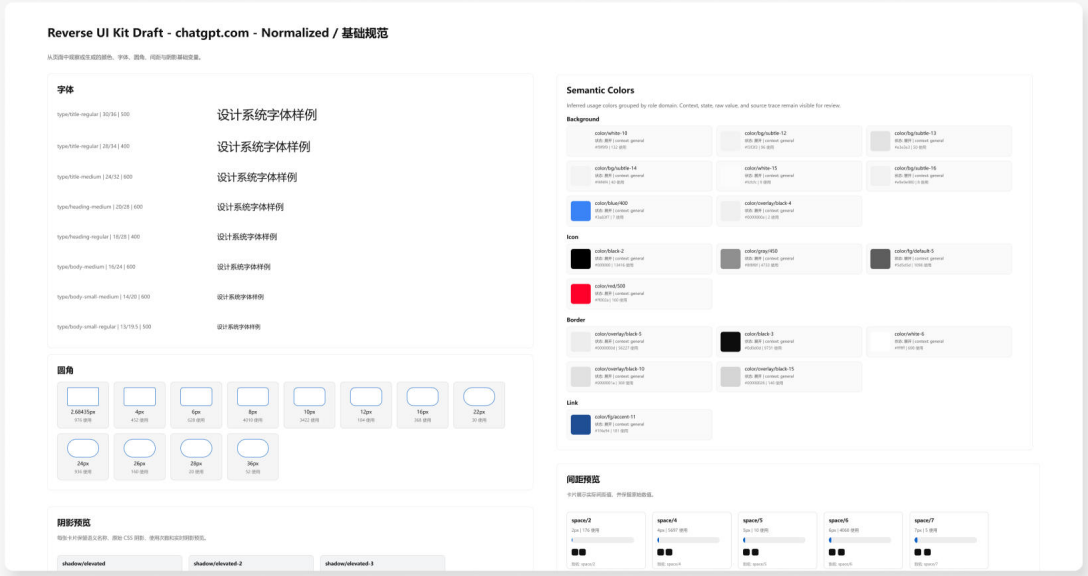
采集完成第一个网页后，按钮恢复为平级的导出和采集，可选择继续抓取或导出已有的内容。

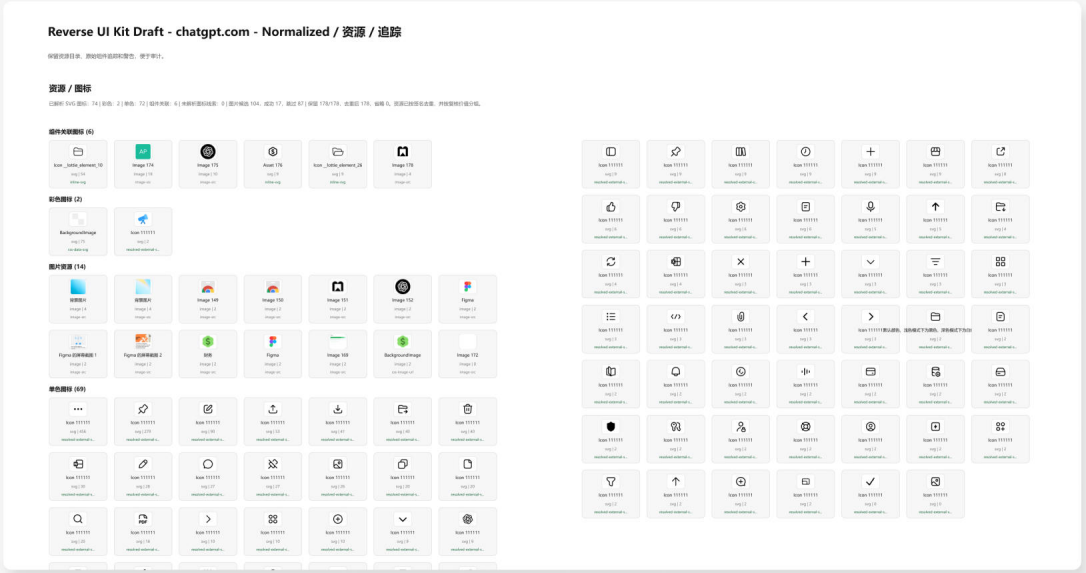
准备离开网页端

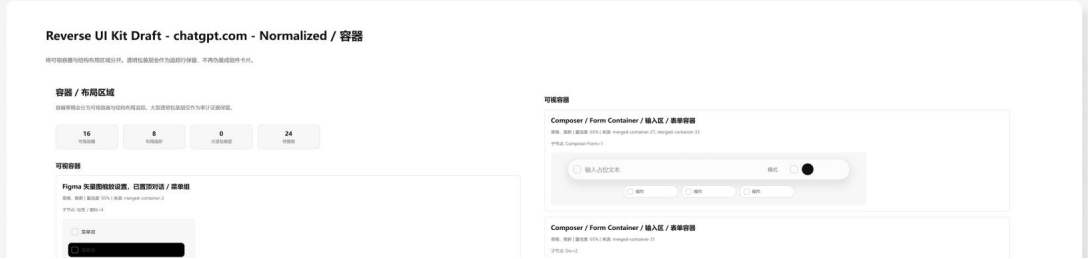
按钮视觉重心变为导出所有界面，引导用户完成批量打包与导出。

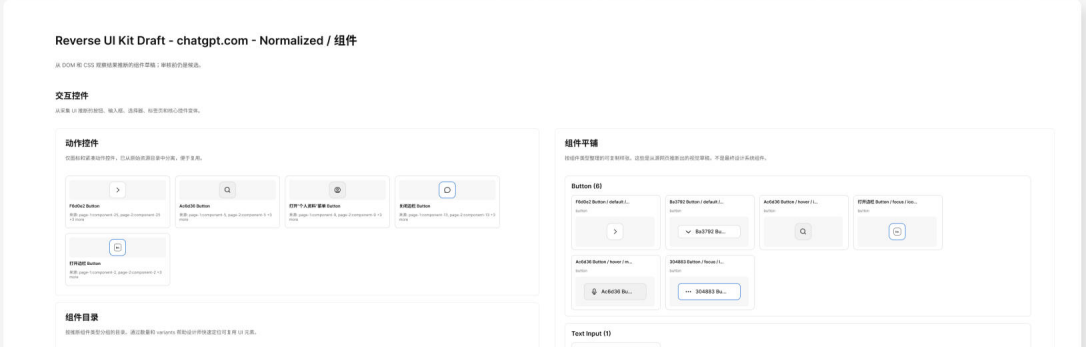
自动整理到这里，最后一步交给设计师。

一份采集数据可以整理出四份草稿。工具负责提取和归类，有设计师来决定最后的稳定内容需要保留哪些。









先把想法跑通，再让真实用户指出问题。

我借助AI完成双端原型验证，并通过开源发布观察陌生用户的首次使用，修正设计图里没有暴露的理解偏差。

截至当前，小红书累计数据

获1W播放；超400点赞、500收藏

排除过于理想化的功能

只在画板上做设计，会忽视技术问题，当我实际跑通后发现，有些设想中的抓取结果是不稳定的。AI帮我缩短了设计和代码的距离，让我能够尽早把实际跑不通的功能降级或砍掉。

修复首次使用的歧义

一位用户给我反馈，把Figma插件端上的提示文案误认为了上传入口，但是实际点击之后只承载了切换主题的功能。于是我在最新版本的插件中及时修改了文案内容。

成为好用的中间环节

有用户反馈，通过插件可以更稳定的整理进Figma，然后将链接丢给Codex让ai接着补全组件，这打破了我一开始想在Figma里做完所有设计的预设。也变相的解决了插件在组件还原稳定性上不足的问题。

找到真正的价值

工具不用非得打大包大揽，提供一份及其规范、人和机器都能读懂的中间结果，本身就是一键有价值的事。

验证链路





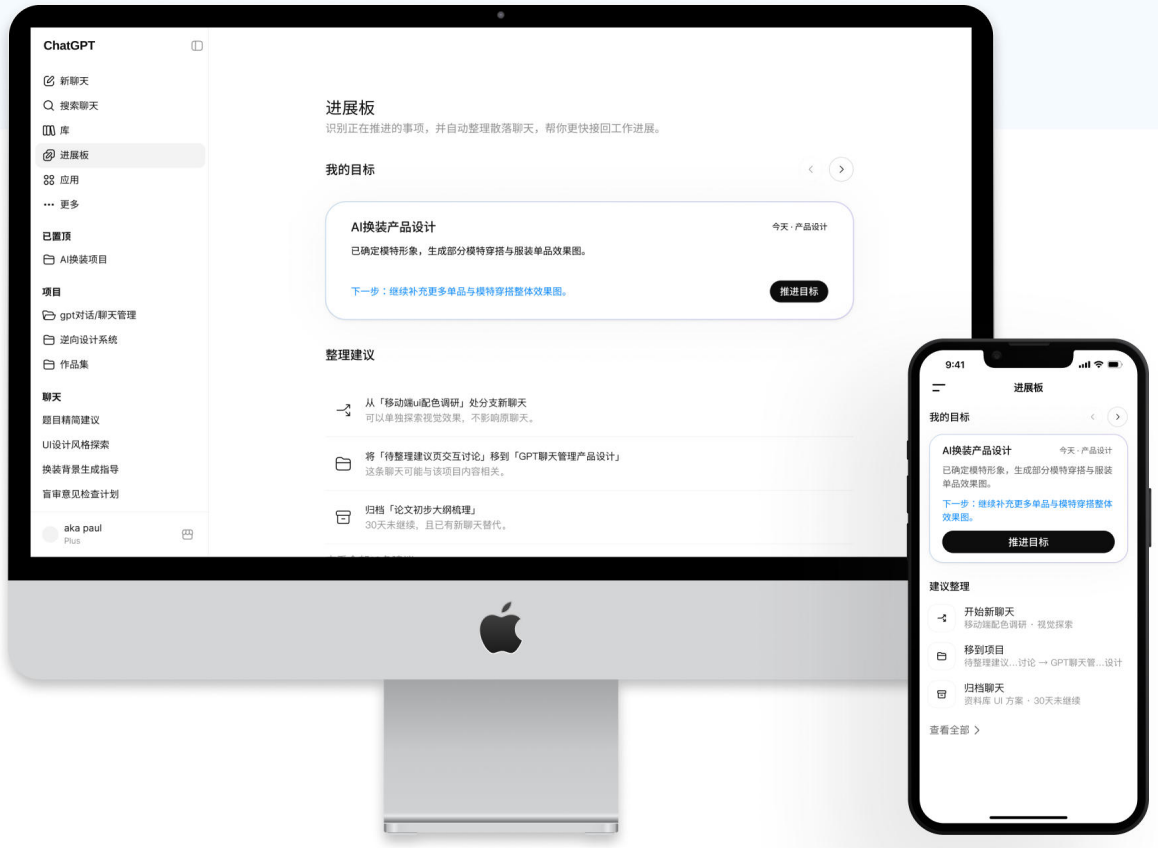
用户反馈：文字存在歧义，误当作入口。



修改结果：更清晰的文字引导。

在原生体验里，
把散落聊天接回正在推进的目标。

我围绕ChatGPT的长期使用场景，重新设计一个更适合街接工作进展的入口。它不替代原有聊天结构，而是在原生框架中增加目标续接与整理建议，帮助用户更快回到下一步。



项目背景

聊天记录越积越多后，用户常在多个分支、阶段性任务和临时想法之间来回切换。真正困难的不是找到某段聊天，而是判断自己正在推进什么，以及下一步该从哪里继续。

设计机会

我没有把它做成新的项目管理工具，而是尽量延续ChatGPT的原生体验，在聊天侧边栏与原始结构中加入进展板入口，让目标续接和聊天整理自然发生。

我的角色与产出

独立完成概念定义、信息架构、Web与移动端高保真设计、关键交互原则梳理与视觉表达，最终输出一套跨端核心界面与作品集展示方案。

聊天越积越多，最耗时的是找回工作状态。

ChatGPT只会按时间线保存文字，不负责理解这些文字背后的进度。
当一个长线任务跨越好几天时，用户就得自己在大脑里充当那根串起所有聊天的线。

聊天线索



一件事会被拆成很多次对话

大部分对话一次聊不完，新的问题不断冒出，会不停产生新的聊天与分支，连续的任务会慢慢散落在不同的地方。

每次回来都需要重新建立上下文

用户往往需要翻出过去的聊天，才能勉强回忆上一步。真正拖慢用户的，是重新恢复进度，想清楚下一步。

“项目”无法告诉你下一步

“项目”功能可以存放同一个项目的文件和对话。但当内容越积越多，用户的下一步该做什么，AI依然没有给答案。

从管理聊天收束到恢复工作状态。

早期方案尝试用话题、快照和详情机制重新组织取落聊天。但功能越详细，用户越需要理解关系、判断建议并维护新的结构。我最终把这些复杂判断重新交给AI，只把真正需要用户行动的结果留在前台。

早期方案：功能越来越完整，判断却没有减少。

① 先判断继续什么

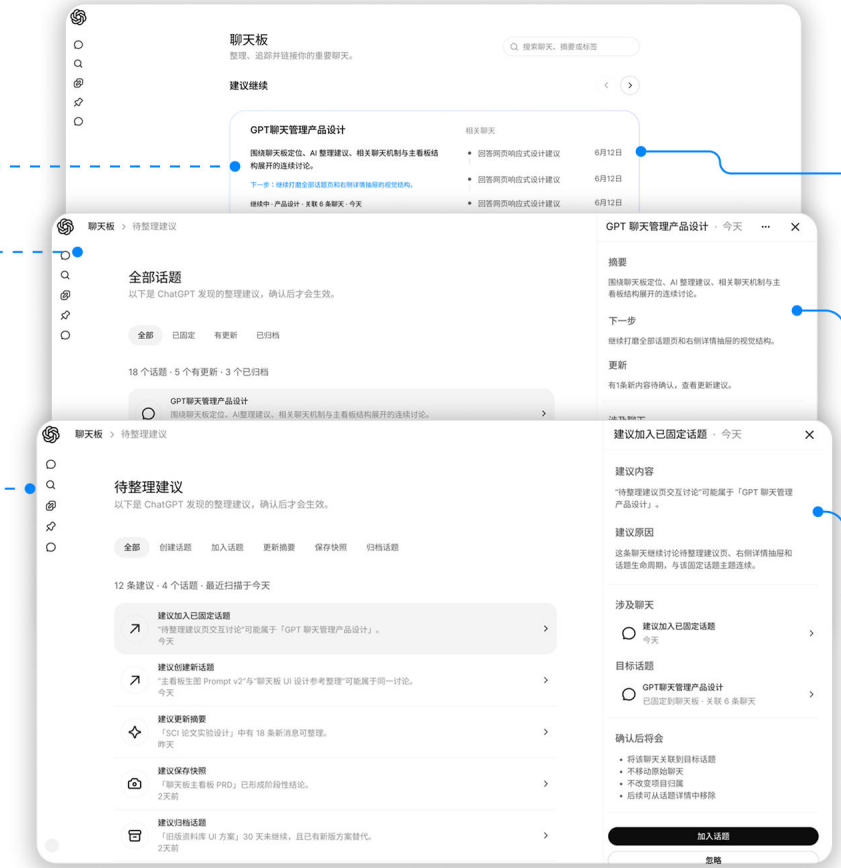
推荐了可能相关的聊天，但用户仍要里新阅读，才能恢复自己做到哪里。

② 再判断 AI 为什么这样整理

为了让建议看起来可信，详情页展示了大量依据，用户反而需要重新理解AI的分析过程。

③ 还要维护新的内容结构

话题、摘要、快朋和关系逐渐形成新的管理层，继续工作之前又多了一套需要理解的规则。

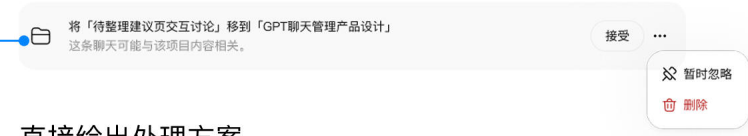


最终收束：复杂理解留在后台，前台只呈现结果



直接恢复工作状态

不再告诉用户应读打开哪条聊天，而是直接说明已经做到哪里，以及现在最值得继缓什么。



直接给出处理方案

AI 往后台完成关系判断，用户看到的是处理对象，方式和简短原因，只需要决定是否接受。

AI 先读懂历史记录，再输出两类可行动结果。

进展板的设计不去改原生GPT聊天的基本形态，它增加的只是让ChatGPT先去理解这些内容彼此之间的关系，再把结论整理成用户立刻就能判断的动作。



一类结果负责继续推进事情

系统读取历史聊天，识别出目标，总结当前进展,最后生成下一步。这会落到「我的目标」卡片里。用户根本不用管背后关联了多少条聊天，只要知道自己现在处于什么阶段就行。

一类结果负责整理零散内容

系统判断聊天之间的关系,生成移动、分支或归档的原生功能建议。每条建议都会带着具体的对象和处理方式，抛给用户做最为简单的判断操作。

真正改变内容前，再把决定交给用户

ChatGPT 可以在后台大胆分析，但绝不会自作主张直接改变聊天的结构。「接受」就是最后一道保险：AI负责出主意，用户决定按不按这个主意办。

推进目标，直接带回原对话

如果下一步只停留在进展板里,它就只是一个备忘录。
点击「推进目标」以后，用户必须能回到原来的上下文，继续之前真正进行的工作。



1 **进入前，先知道今天要做什么**

目标卡先告诉用户现在做到哪里、接下来要做什么。点进去以前，用户已经重新知道了当前工作的方向。

2 **进入后，保留真正有用的上下文**

推进目标不会再打开一套新的任务页面。用户回到的依然是之前工作的ChatGPT对话，过去的讨论和结论都继续保留。

3 **下一步直接接到新的输入上**

进展板里生成的下一步会继续出现在输入区域。从重新想“今天做什么”，到直接开始工作，中间只需要一次点击。

4 **提供原路返回的出口**

在对话中直接提供回到进展版页面的出口，与ChatGPT的原生操作一致，减轻用户额外的理解负担。

“整理建议”直接给出方案，用户只做建议。

面对越堆越乱的历史聊天，光提醒用户去整理是没有用的。
AI必须直接告诉用户：处理哪一条，怎么处理，将用户的负担降到最低。

一条建议就是一个完整判断

每条建议已包含处理结果，如移到项目、开启新聊天、归档等，并给出简要原因，用户可一眼判断是否符合预期。





处理过的建议仍然有状态保留

点过的「已接受」和「已忽略」不会凭空消失，而是保留成独立的分类。用户处理完以后，心里依然有底，知道自己之前对哪些建议做过决定。

一键批准整批判断

页面里的每条建议动作都不一样，但用户不需要接个去点。用户接受的是ChatGPT已经分别做好的这批判断，一键确认后，系统就批量操作。

鼠标悬停后再出现操作按钮

鼠标移到某一条建议时，这一行会用原生hover状态高亮，下方才顺势滑出操作按钮。用户的视线在哪里，操作才在哪里出现。

沿用原生视觉规范，
将核心信息通过一屏展示。

虽然加了新内容，但我没有去设计一套全新的视觉语言。而是通过项目二的逆向设计系统，直接借用ChatGPT现成的设计语言，通过原生的颜色、圆角和留白等，让新功能页像是原生自带的一样。

双端一致承载两类任务

移动端承载与桌面端一致的两类任务。上部分“我的目标”在操作中可以清晰掌握目标详情。下部分“整理建议”可以快速查看最近待整理，需集中处理在进入详情页。

“建议整理”重新适配竖屏阅读

移动端将建议整理的文案进一步精简，建议列表改成更适合竖屏浏览的上下两层结构，让单手浏览和识别更轻松。

主动作在不同页面保持同一等级

「推进目标」和「全部接受」用的都是最明确的主按钮。一个是让你接着干活，一个是让你确认AI的整体判断，用户不需要重新去判断哪个按钮最重要。



AI越会判断，界面越应该把下一步说简单。

进展板经历过一轮复杂化，最后重新回到简单。
我最后得到的认识是：与其让界面展示AI 有多聪明，不如让AI的判断减少用户需要处理的事情。



复杂分析全部留在后台

识别聊天属于哪个目标、判断进行到了哪里、决定移动还是归档，这些分析都交给模型在后台完成。用户不需要阅读完整的判断过程。

用户只看能马上使用的结果

用户真正需要看到的是：当前进展、下一步、推进目标和接受建议。界面中的信息最终都应该帮助用户做出清楚的下一步动作。

“进展板”负责续接，“聊天继续”负责工作

真正的讨论、写作和研究依然发生在原来的对话里。进展板只增加一层轻量状态，让用户每次回来都知道自己做到哪里，以及接下来从哪里继续。

在不确定中建立可控的创作流程，让AI成为可靠的创意伙伴。

用户走了弯路，
但“弯”在哪里并不容易判断。

外卖、到餐和酒店等复杂移动业务中，同一个目标往往存在多条完成路径。完成率和任务时长能够判断任务是否顺利，却很难解释用户究竟在哪里发生绕行、重复或反复尝试。因此，我们尝试从真实操作路径中识别这些异常行为，建立一套用于体验诊断的移动界面动线冗余评价方法。

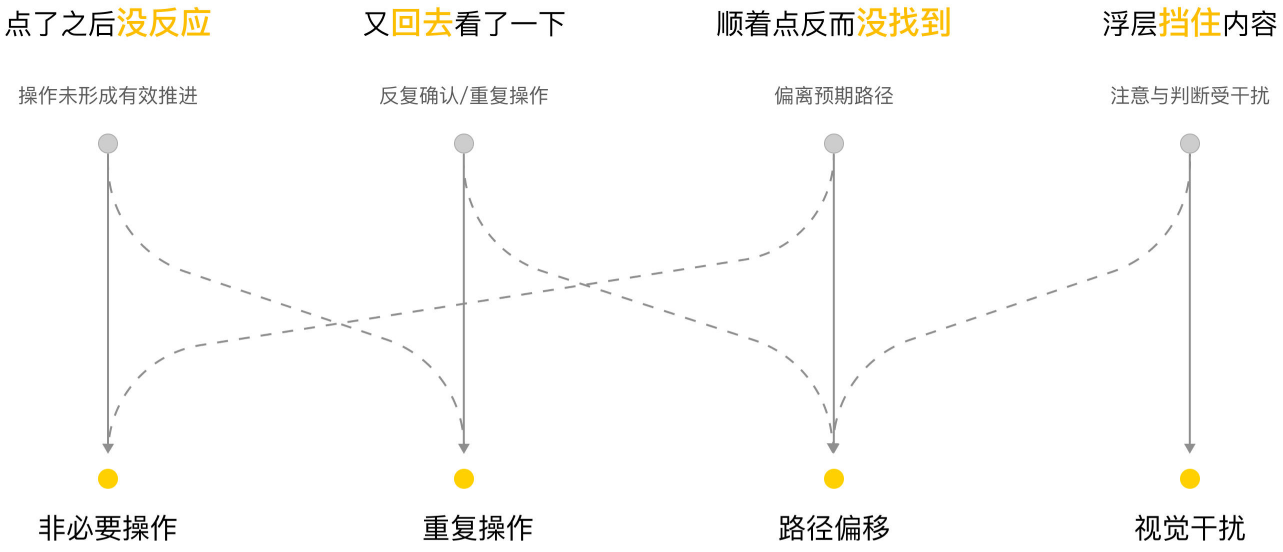
任务示例 | 完成指定外卖套餐下单



在定义“冗余”之前，
先看用户到底在哪里觉得“绕”。

我们没有直接询问“界面是否复杂”，而是把问题放进真实任务：先观察用户如何完成目标，再通过事后回顾理解停顿、返回、误点与额外探索为什么发生。

用户感知 → 行为抽象



4 类行为方向进一步拆解为 11 类候选冗余行为

我的角色

用户研究与实验参与·行为量化与程序验证

40+ | 用户参与

11 | 典型任务

外卖 / 到餐 / 酒店

任务执行 + 事后回顾

15 条半结构式访谈问题

参与方式

用户先完成任务，再回过头解释自己为什么这样操作。40 余名参与者分别在外卖、到餐和酒店场景中完成 11 个典型任务。任务结束后，通过事后回顾继续追问过程中出现的停顿、返回、误点、重复确认、路径偏移与额外探索，从用户的自然表达中寻找反复出现的行为模式。

把用户感受到的“绕”，转成机器能识别的行为。

基于交互日志重建真实路径，识别冗余行为模式，形成可计算的四个核心指标。

11类候选行为 ⟶ 专家判断 x 行为验证 ⟶ **4个核心指标**

原始交互事件
埋点拿到的是零散的操作记录。

重建行为路径
进一步清洗并整理操作记录。

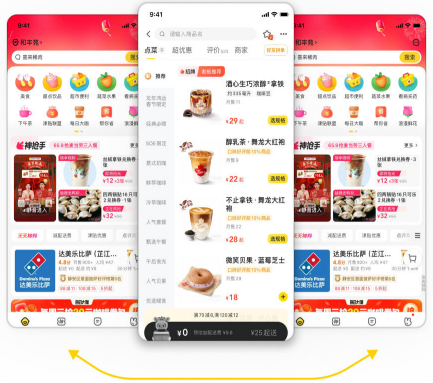
识别行为模式
匹配页面模式并判定行为。

输出可计算指标
形成可用于量化分析的结果。

Session示例

事件	页面/控件	时间戳
Page	美团首页	10:20:50
Click	外卖入口	10:21:00
Page	商家详情页	10:21:07
Click	详情入口	10:21:20
Page	商品详情页	10:21:55
Click	下单入口	10:22:12
Page	提单页	10:22:41

美团首页 → 商品列表页 → 美团首页



返回原界面

页面序列：A → B → A

识别示例

重复回退 非必要操作

进入页面后又返回上一页

匹配规则

检测连续页面模式：A→B→A，统计出现次数。

计算示例

重复回退（BR）

往返次数 / 页面跳转总次数

该指标体验得分

2 / 18

BR = 0.111

假定该任务下页面跳转总次数为：18次

所有展示内容均为脱密模糊示例，仅展示行为建模思路。

识别异常还不够，还要知道它什么时候开始影响体验。

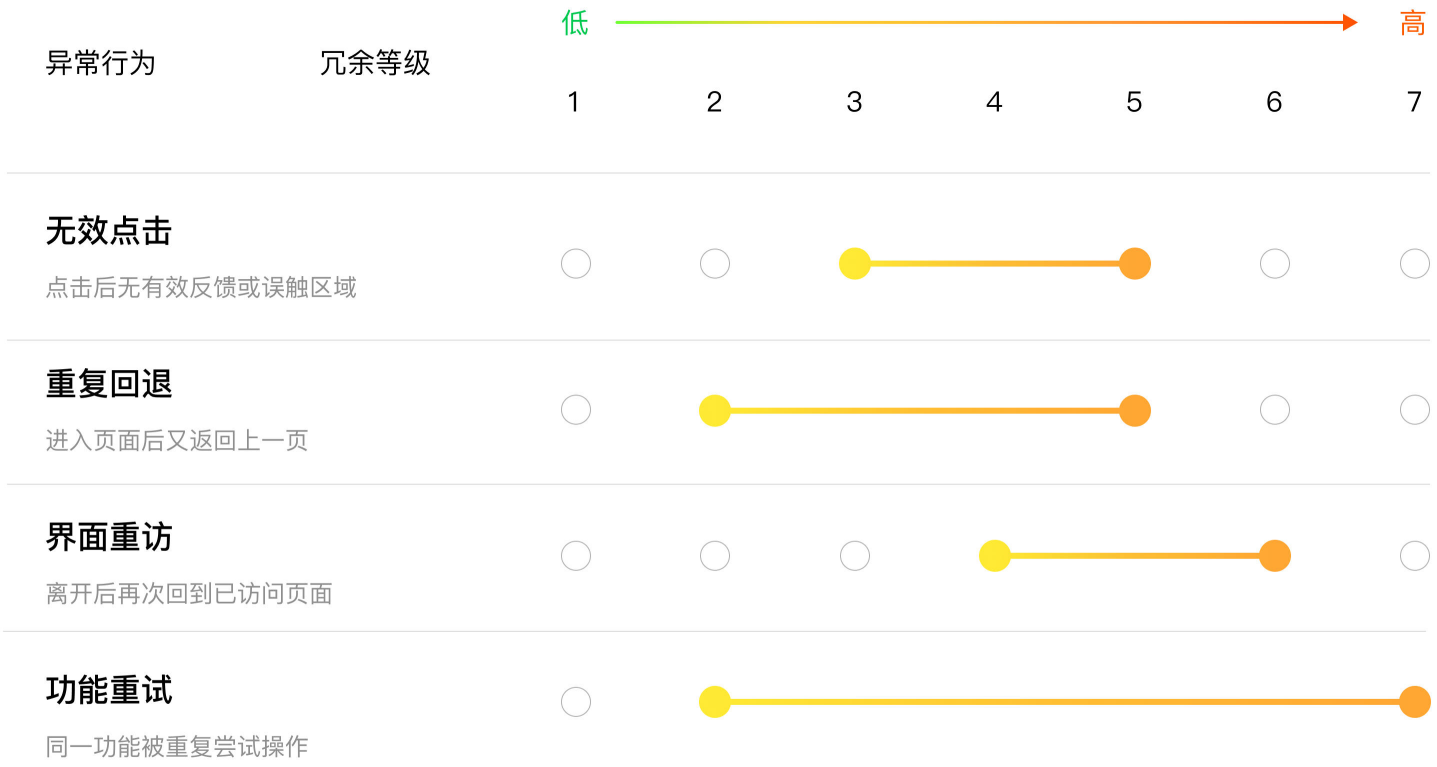
我们通过用户实验找到四类冗余行为的体验边界，并将它们映射到同一体验尺度,形成可比较的整体评价。

35 名用户
逐项标定体验边界

40 名用户
验证多指标组合体验

四类异常行为在7个冗余水平下的体验边界(示例)

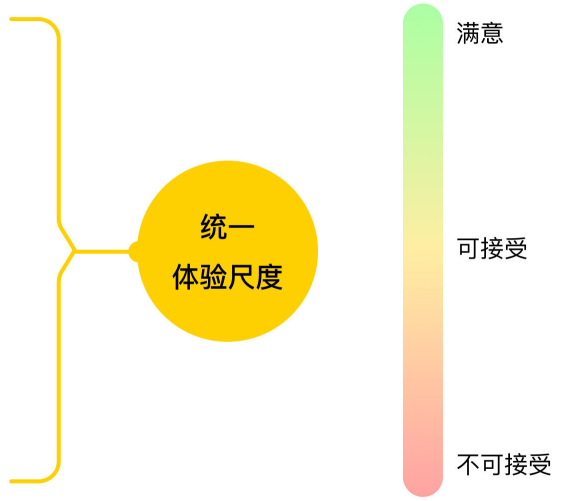
用户可接受边界　用户不可接受边界



将四类异常行为映射到统一体验尺度

不同异常达到体验拐点的位置不同，因此不能再原始尺度上直接比较。

整合后用于整体冗余评价，支撑优化优先级判断。



量化的终点，
是让问题重新回到具体界面。

将评估方法放回真实业务数据之后，我们的目标不只是发现异常，而是让这些异常指向值得检查的具体界面问题。

200+
条真实案例验证

86.7%
验证归因正确率达

Precision 71.1% | Scoped Recall 75.0%

偏离路径，不一定意味着用户做错了

比较、探索、主动确认等行为也会产生额外操作，因此量化结果只揭示了高暴露点、指向值得检查的地方，无法在没有上下文的情况下自动解释用户意图。

部分异常行为	问题线索	设计检查
重复回退 进入页面后又返回上一页	入口预期与目标内容可能不一致	入口语义检查文档
界面重访 离开后再次回到已访问页面	用户需要跨页面重新确认信息	信息分布检查文档

数据负责暴露异常，设计判断负责解释异常。

仅展示部分核心映射关系，均做脱密模糊处理。

感谢观看，完结撒花。

让我加入你们吧，我们一起把产品的用户体验做得更加美妙！

Zhangyong

手机/微信：130-7543-4960　　邮箱：zhangyong@njfu.edu.cn